

Peoples Bakers — Advanced Technical Project Guide

Full-stack bakery e-commerce platform · Architecture, API, security, deployment & QA

Version: 2.0 Advanced | **Date:** June 2026 | **Stack:** Vanilla JS + Node.js 22.5+ + Express 4 + SQLite (node:sqlite)

Audience: Developers, technical interviewers, DevOps, and stakeholders who need a production-grade understanding of the system — not just how to run it locally, but how it is designed, secured, and deployed.

Table of Contents

1. Executive Summary	11. Admin Dashboard
2. System Architecture	12. Internationalization (i18n)
3. Technology Stack	13. PWA, Caching & Performance
4. Repository Structure	14. Mobile Responsive Design
5. Local Development Setup	15. Security Audit & Bug Fixes
6. Application Routes & URLs	16. Environment Configuration
7. REST API Reference	17. Testing & Quality Assurance
8. Authentication & Authorization	18. Deployment & Hosting
9. Order & Payment Lifecycle	19. Troubleshooting
10. Database Layer (SQLite)	20. Quick Reference

1. Executive Summary

Peoples Bakers is a full-stack web application for a Sri Lankan bakery business. Customers browse products, build a cart, place orders (WhatsApp, cash-on-delivery, or PayHere online payment), manage accounts, submit feedback, and locate outlets on a map. Staff use a protected admin dashboard for order fulfilment, catalog management, and analytics.

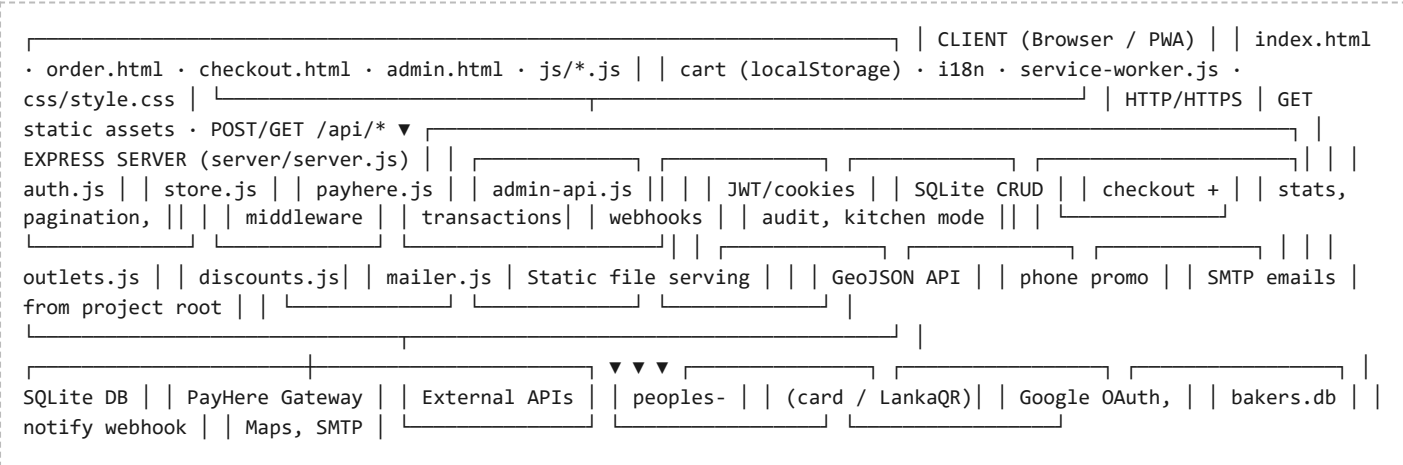
Key characteristics

Aspect	Detail
Architecture	Monolithic Express server serving static frontend + JSON REST API
Frontend	Multi-page application (MPA) — HTML/CSS/vanilla JS, no React/Vue build step
Backend	Node.js 22.5+ with Express 4, modular route handlers
Persistence	SQLite file via Node built-in <code>node:sqlite</code> — no MySQL/Postgres required
Auth	JWT in httpOnly cookie (<code>pb_token</code>), bcrypt password hashing, optional Google OAuth
Payments	PayHere (Sri Lanka) with server-side hash verification and webhook notify URL
Languages	English, Sinhala (සිංහල), Tamil (தமிழ்) — client-side i18n with JSON dictionaries
Mobile	Responsive CSS (30+ breakpoints), hamburger nav, progressive image loading

Critical constraint: This is *not* a static site. Opening HTML files directly (`file://`), VS Code Live Server, GitHub Pages, or PHP-only hosting will break login, orders, admin, and payments. A Node.js process must run continuously in production.

2. System Architecture

2.1 High-level diagram



2.2 Frontend vs backend responsibilities

Layer	Responsibility	Cannot do alone
Frontend	UI rendering, cart state (localStorage), form validation, language switching, map display, PayHere redirect	Persist orders, authenticate users, enforce prices, process payments, admin data
Backend	Serve files, validate & price orders, JWT auth, SQLite persistence, PayHere hash/webhook, rate limits, admin RBAC	Replace responsive UI — frontend still required for customer experience

2.3 Typical checkout request flow

User fills checkout form (checkout-page.js) | ▼ POST /api/orders → Server validates items against catalog prices | Applies phone discount (logged-in + matching phone only) | Creates order record (status: pending / awaiting_payment) ▼ [COD / WhatsApp] → Order confirmed · optional SMTP email · admin notified | [PayHere] → POST /api/payments/payhere/checkout → Returns signed checkout payload | Browser redirects to PayHere ▼ PayHere POST /api/payments/payhere/notify → Hash verified · paymentStatus = paid

3. Technology Stack

Category	Technology	Version / Notes
Runtime	Node.js	>= 22.5.0 (uses built-in <code>node:sqlite</code>)
HTTP framework	Express	^4.19 — middleware, routing, static files
Database	SQLite	WAL mode; document-style JSON in TEXT columns
Password hashing	bcryptjs	Registration & login
Tokens	jsonwebtoken	7-day expiry; httpOnly cookie
Email	nodemailer	Optional SMTP (Gmail app password supported)
Google login	google-auth-library	Optional; verify ID token server-side
Payments	PayHere REST + redirect	MD5 hash; sandbox/live via env
Frontend	HTML5, CSS3, ES modules	No bundler; cache-busted query strings
PWA	service-worker.js + manifest	Network-first for critical JS/CSS
PDF generation	puppeteer-core + Edge	Dev tooling only (this guide)

4. Repository Structure

```
Peoples-Bakers-master/
├─ index.html, order.html, products.html, account.html, login.html
├─ checkout.html, feedback.html, find-us.html, special-cake.html, cakes.html
├─ admin.html, pb-office.html           ← Admin UI (same app; secret URL configurable)
├─ css/                                ← style.css (~13k lines), admin.css, account.css
├─ js/
│   └─ main.js                         ← Site-wide: cart hooks, mobile nav, progressive images
│   └─ cart.js, checkout-page.js, order-page.js, account-page.js
│   └─ auth.js, admin.js, i18n.js, find-us.js
│   └─ translations/                  ← en.json, si.json, ta.json
├─ images/, icons/, data/outlets.json
├─ server/
│   └─ server.js                      ← Entry point (npm start)
│   └─ auth.js, store.js, seed.js, payhere.js, mailer.js
│   └─ admin-api.js, outlets.js, discounts.js
│   └─ data/peoples-bakers.db         ← Auto-created; git-ignored
├─ scripts/verify-backend.js          ← Automated health checks
├─ docs/                              ← Hosting guides + this PDF source
├─ render.yaml                       ← Render.com deploy blueprint
├─ .env / .env.example                ← Secrets (never commit .env)
└─ package.json
```

5. Local Development Setup

Prerequisites

- Node.js 22.5 or newer — nodejs.org
- Git or ZIP of project folder
- Terminal (PowerShell on Windows)

Commands

```
cd "C:\path\to\Peoples-Bakers-master"
npm install          # First time only – installs node_modules
npm start           # Starts Express on http://localhost:3000
npm run dev         # Auto-restart on file changes (Node --watch)
npm run verify      # Health checks (server must be running)
npm run seed        # Re-seed products/admin (no duplicates)
npm run promote-admin -- user@email.com
```

Success indicator: Terminal prints Peoples Bakers server running at `http://localhost:3000`. Keep the terminal open — closing it stops the server. Use **Ctrl+C** to stop intentionally.

First-run seed data

- Admin account created automatically (credentials printed in terminal)
- Default dev: `admin@peoplesbakers.lk` / `admin123` — change before production
- Products imported from seed data into SQLite

6. Application Routes & URLs

URL	Page	Description
/	index.html	Home — hero carousel, featured products, outlets
/order.html	Order Online	Delivery zones, FAQ, WhatsApp order, cart link
/checkout.html	Checkout	Order submission, PayHere, phone discount
/products.html	Catalogue	Full product & cake listing
/account.html	My Account	Profile, order history (auth required)
/login.html	Auth	Register, login, Google Sign-In (optional)
/find-us.html	Outlet finder	Map, search, nearby branches
/feedback.html	Feedback	Customer reviews → POST /api/feedback
/special-cake.html	Custom cakes	Photo cake requests via WhatsApp
/pb-office.html	Admin	Staff dashboard (configurable via ADMIN_PATH)
/admin.html	Admin (alt)	Same dashboard — both URLs supported
/api/health	API	JSON health check for monitoring
/api/config	API	Public site config (WhatsApp, phone, social URLs)

7. REST API Reference

7.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/register	—	Create customer account (name, email, password, phone)
POST	/api/auth/login	—	Sets httpOnly JWT cookie
POST	/api/auth/logout	—	Clears cookie
GET	/api/auth/me	Cookie	Current user profile
PATCH	/api/auth/profile	User	Update name, phone
POST	/api/auth/google	—	Google ID token → account + cookie

7.2 Orders & payments

Method	Endpoint	Auth	Description
POST	/api/orders	Optional	Create order; server validates catalog & prices
GET	/api/orders	User	Own orders (admin sees all)
GET	/api/orders/:id	User	Single order detail
PATCH	/api/orders/:id	Admin	Update status, admin notes
POST	/api/orders/:id/cancel	User/Admin	Cancel order

POST	/api/payments/payhere/checkout	—	Generate PayHere signed checkout
POST	/api/payments/payhere/notify	PayHere	Webhook — updates paymentStatus

Order statuses: pending → awaiting_payment → confirmed → preparing → delivered | cancelled

Payment statuses: cod, pending, paid, failed, cancelled, refund_pending

7.3 Products, feedback, outlets, config

Method	Endpoint	Auth	Description
GET	/api/products	—	Public product catalogue
POST/PUT/DELETE	/api/products[:id]	Admin	CRUD products
POST	/api/feedback	—	Submit review (rate-limited)
GET/PATCH	/api/feedback[:id]	Admin	List / moderate feedback
GET	/api/outlets, /nearby, /bounds, /geojson	—	Branch finder APIs
GET	/api/discount/preview	—	Preview phone discount eligibility
GET	/api/config	—	Runtime public configuration

7.4 Admin API (prefix /api/admin/*)

Mounted via `admin-api.js`. All routes require `requireAuth` + `requireAdmin`.

- Dashboard stats, revenue analytics, order volume
- Paginated orders with search, status filters, date range
- User management, settings (WhatsApp, business name, social links)
- Audit trail for admin actions
- Kitchen display mode endpoints

8. Authentication & Authorization

8.1 JWT cookie model

- Cookie name: `pb_token`
- Flags: `httpOnly`, `sameSite: lax`, `secure: true` in production
- Expiry: 7 days
- Payload: user id, name, email, phone, role (customer | admin)

8.2 Role-based access

Role	Access
customer	Own profile, own orders, public pages
admin	All admin API routes, order management, products, feedback, users, settings

8.3 Security hardening (implemented)

- **Production JWT_SECRET:** Minimum 32 characters required; server exits if missing
- **Stale JWT role fix:** Middleware re-loads user from DB on each request — promoted/demoted roles apply immediately
- **Admin seed password:** `ADMIN_SEED_PASSWORD` required in production (no default admin123)
- **Secret admin URL:** `ADMIN_PATH` env (default pb-office.html) — not linked from public nav
- **Server-side pricing:** Client-submitted prices ignored; catalog is source of truth
- **Phone discount:** Only applied when logged-in user's phone matches checkout phone

9. Order & Payment Lifecycle

9.1 Order creation

1. Frontend sends cart items + customer details to `POST /api/orders`
2. Server maps product IDs to catalog entries; rejects unknown or zero-qty items
3. Discount calculated server-side (`discounts.js`)
4. Order persisted with timestamps, payment method, delivery mode
5. Optional confirmation email via `mailer.js`

9.2 PayHere integration

- Checkout endpoint returns merchant_id, hash, amount, return/cancel/notify URLs
- User completes payment on PayHere hosted page
- PayHere POSTs to `/api/payments/payhere/notify` — MD5 hash verified server-side
- Failed payment no longer auto-cancels order (fixed) — staff can follow up manually
- Production defaults to **live** mode unless `PAYHERE_SANDBOX=true`

10. Database Layer (SQLite)

10.1 Design pattern

Collections: `users`, `orders`, `feedback`, `products`, `settings`. Each table: (id TEXT PK, createdAt TEXT, doc TEXT) where `doc` is JSON. This document-store pattern keeps route code simple while using a real SQL database.

10.2 Operations

- WAL journal mode for concurrent read performance
- Write transactions for crash-safe multi-step updates
- Legacy JSON files auto-migrated on first run
- Override path: `DB_PATH=/persistent/peoples-bakers.db`

10.3 Reset

Stop server → delete server/data/ folder → npm start (re-seeds)

11. Admin Dashboard

Accessible at `/pb-office.html` (or custom `ADMIN_PATH`). Frontend: `admin.html` + `js/admin.js` + `css/admin.css`.

Feature	Capability
Orders	Filter, search, paginate; update status pipeline; admin notes
Products	Add, edit, delete catalogue items synced to <code>/api/products</code>
Feedback	Read and moderate customer submissions
Users	View registered customers
Settings	WhatsApp number, phone, business info, social URLs
Analytics	Revenue, order counts, status breakdown
Kitchen mode	Simplified display for fulfilment screens

12. Internationalization (i18n)

- Engine: `js/i18n.js` loads `js/translations/{en,si,ta}.json`
- Markup: `data-i18n="section.key"` on HTML elements
- Switcher: EN | සිං | து — preference stored in `localStorage`
- Coverage: navigation, footer, home, order, account, checkout strings, mobile menu labels

13. PWA, Caching & Performance

- **service-worker.js**: Caches static assets; network-first strategy for main JS/CSS (prevents stale code)
- **Cache versioning**: Bumped on releases (e.g. `peoples-bakers-v147`)
- **Asset query strings**: CSS/JS `?v=N` for cache busting
- **Config API**: Contact numbers loaded dynamically — single source of truth

14. Mobile Responsive Design

Majority of Sri Lankan e-commerce traffic is mobile. The site implements responsive layout, touch-friendly controls, and network-aware image loading.

14.1 CSS breakpoints

`css/style.css` includes 30+ media queries at 768px, 900px, 1024px, etc. Grids collapse to single column on small screens.

14.2 Hamburger navigation ($\leq 900\text{px}$)

- `js/main.js` — `setupMobileNav()`
- Side drawer with backdrop; closes on link click, backdrop tap, Escape key
- `body.nav-open { overflow: hidden }` prevents background scroll

14.3 Progressive image loading (3G / slow networks)

Technique	Implementation
-----------	----------------

Hero defer	Only first slide loads immediately; others staggered
Hero preload	First hero image preloaded for faster LCP
Lazy images	<code>loading="lazy"</code> on below-fold images
Product backgrounds	<code>setupProgressiveImages()</code> — IntersectionObserver-based

15. Security Audit & Bug Fixes

15.1 Critical FIXED

Issue	Resolution
Stale cart at checkout	Re-read cart + proper event binding before submit
Client-side price tampering	Server catalog prices only; invalid products rejected
Default admin password in prod	ADMIN_SEED_PASSWORD required when NODE_ENV=production
Weak JWT secret	32+ char JWT_SECRET enforced in production

15.2 Medium FIXED

Issue	Resolution
Admin URL 404	Both /pb-office.html and /admin.html served
Stale JWT role	DB user lookup on each authenticated request
PayHere sandbox default	Production uses live unless PAYHERE_SANDBOX=true
Payment fail cancels order	Order retained for manual follow-up
Phone discount fraud	Logged-in phone must match checkout phone
Wrong WhatsApp numbers	Centralized via /api/config; default 947228955477
Service worker stale cache	Network-first for critical assets

15.3 Low priority FIXED

- Social links (Instagram, LinkedIn) — real URLs + config API
- Full i18n on home, order, account pages
- Feedback rate limiting
- SQLite write transactions
- Outlets catalog graceful fallback
- Mobile hamburger menu + progressive images

16. Environment Configuration

Copy `.env.example` → `.env`. Never commit `.env` to version control.

Variable	Required	Purpose
JWT_SECRET	Production: yes	Sign login tokens (32+ random chars)
NODE_ENV	Production: yes	production → secure cookies, stricter checks
SITE_URL	Recommended	Email links, PayHere return URLs
ADMIN_SEED_PASSWORD	Production: yes	Initial admin account password
ADMIN_PATH	Optional	Secret staff URL filename
WHATSAPP_NUMBER	Optional	Default: 947228955477
PAYHERE_*	For online pay	Merchant ID, secret, sandbox flag

SMTP_*	Optional	Order confirmation emails
GOOGLE_CLIENT_ID	Optional	Google Sign-In
DB_PATH	Optional	Persistent disk path on host
PHONE_DISCOUNT_*	Optional	Checkout phone promo rules

17. Testing & Quality Assurance

17.1 Automated verification

```
npm start           # Terminal 1 – keep running
npm run verify      # Terminal 2 – runs scripts/verify-backend.js
```

Checks: /api/health, /api/config, /api/outlets, admin auth, static assets.

17.2 Manual test checklist

- 1. Home page — images, language switch (EN → தமிழ் → தமிழ்)
- 2. Products → add to cart → checkout (COD path)
- 3. Register → login → account order history
- 4. Admin login → view/update orders
- 5. WhatsApp buttons — verify +94 22 895 5477
- 6. PayHere sandbox checkout (if keys configured)
- 7. Mobile: hamburger menu, checkout on phone keyboard
- 8. Hard refresh Ctrl+Shift+R after deploy

18. Deployment & Hosting

Companion document: Full hosting checklist, provider comparison, and Mobitel inquiry template — see [docs/Peoples-Bakers-Hosting-Guide.pdf](#) (English) or [Peoples-Bakers-Hosting-Guide-Sinhala.pdf](#) on Desktop.

18.1 Requirements

Requirement	Detail
Runtime	Node.js 22.5+ running 24/7
Start command	<code>npm start</code>
Storage	Persistent disk for SQLite (~1 GB)
SSL/HTTPS	Required — login cookies, PayHere webhooks
RAM	512 MB min, 1 GB recommended

18.2 Compatible hosts

Option	Suitable	Notes
Render.com	✓ Best	render.yaml included; Singapore region
VPS (DO, Linode)	✓ Good	Full control; PM2/systemd for process
Railway / Fly.io	✓ Good	Node-native PaaS
PHP-only cPanel	X No	Cannot run Express backend
GitHub Pages	X No	Static only — no API

18.3 Production vs local

Locally you run `npm start` manually. On a live host, the platform keeps Node running automatically — customers never need your laptop online. Set all production env vars on the host dashboard before go-live.

19. Troubleshooting

Symptom	Cause	Fix
Login/API fails	Opened HTML directly or Live Server	Use <code>npm start</code> → localhost:3000
Site stops working	Terminal closed	Restart <code>npm start</code> ; production: check host logs
Port 3000 in use	Another process	Close old terminal or <code>PORT=3001 npm start</code>
Server exits on start	Missing JWT_SECRET in production	Set <code>.env</code> on host
Old UI after update	Browser or SW cache	Ctrl+Shift+R; clear site data
PayHere notify fails	HTTP not HTTPS	Enable SSL; notify URL must be public
Orders lost on restart	Ephemeral disk on host	Set <code>DB_PATH</code> to persistent volume

20. Quick Reference

Task	Command / URL
Install dependencies	<code>npm install</code>
Start server	<code>npm start</code>
Website	<code>http://localhost:3000</code>
Admin panel	<code>http://localhost:3000/pb-office.html</code>
Health check	<code>GET /api/health</code>
Verify script	<code>npm run verify</code>
Database file	<code>server/data/peoples-bakers.db</code>
Default WhatsApp	+94 22 895 5477 (947228955477)
Mobile nav breakpoint	900px — hamburger menu
Stop server	Ctrl+C in terminal

For interviews / handover: Emphasize server-side price validation, JWT httpOnly cookies, PayHere webhook verification, SQLite persistence on hosted disks, and the requirement for Node.js hosting — these distinguish this from a static bakery brochure site.